FACULDADE DE
CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE FÍSICA

Machine Leaning (ML)

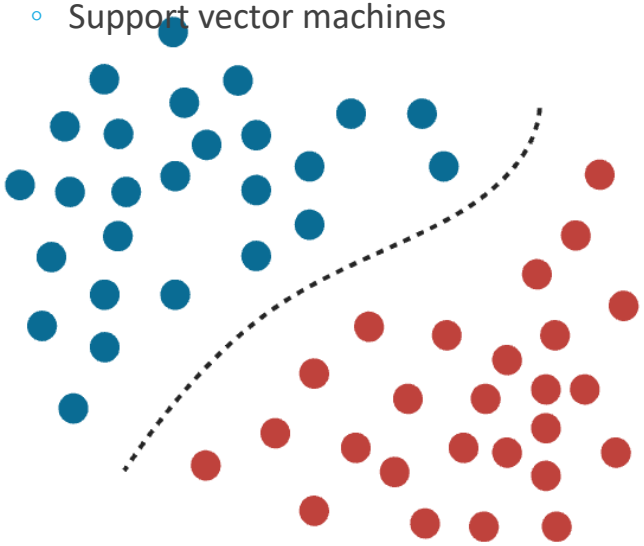
DISCRIMINATIVE MODELS – DECISION TREES, KNN & SVM

Pedro Vieira
pmv@fct.unl.pt

Discriminative vs Generative

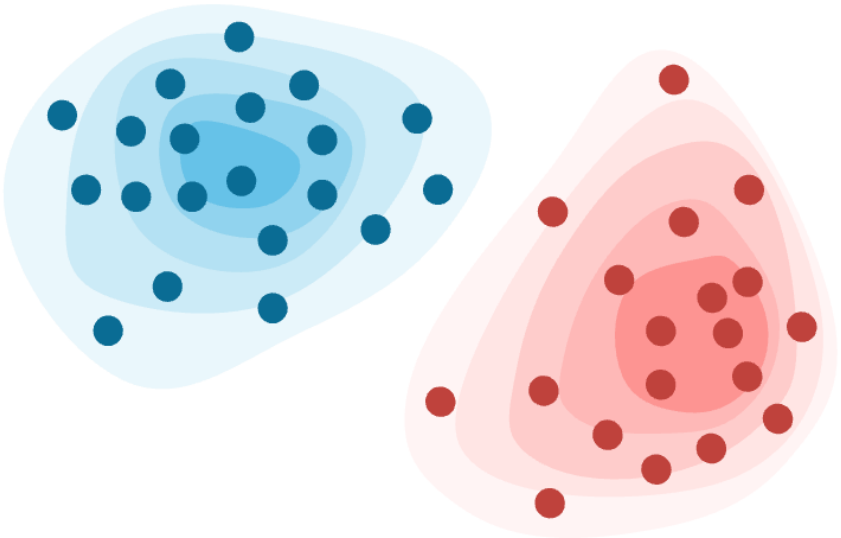
Discriminative Model:

- Predicts a label/class given the features of input data
  - Linear and Logistic regression
  - Decision tree's
  - K nearest neighbours (KNN)
  - Support vector machines



Generative Model

- Learns the data-generation process by modelling each class's probability distribution
  - Bayes
  - Naive Bayes
  - Gaussian Discriminant Analysis (GDA)

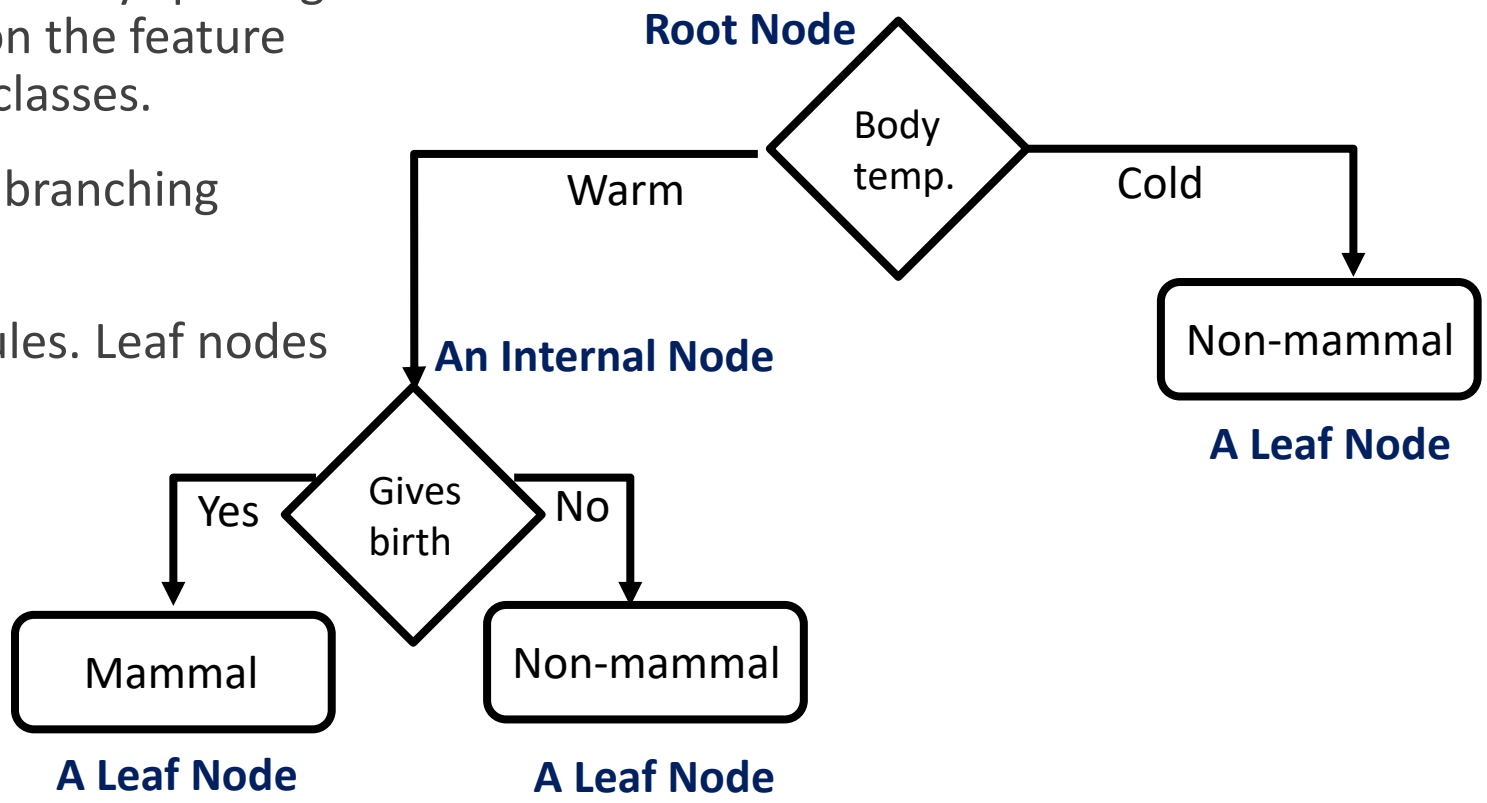


Decision Tree

A decision tree is a supervised learning model that makes predictions by repeatedly splitting the data into branches based on the feature that best separates the target classes.

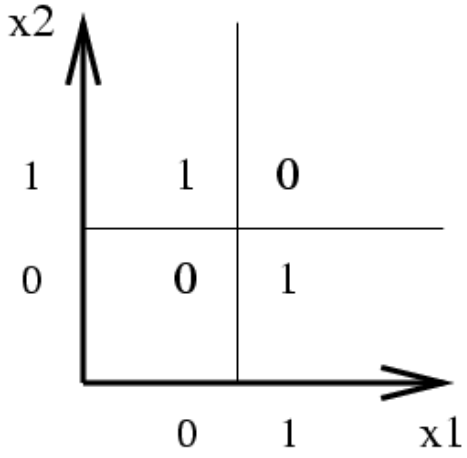
A predictive model based on a branching series of Boolean tests

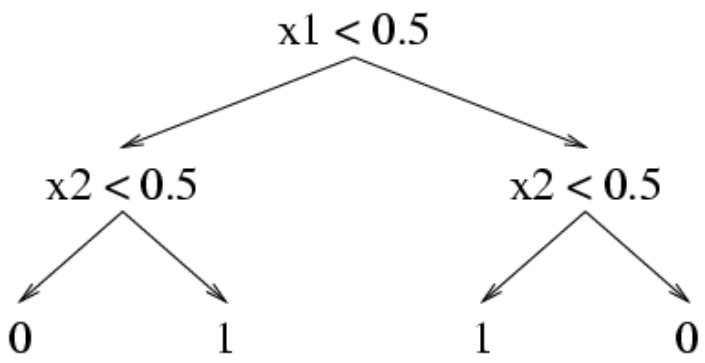
Root and internal nodes test rules. Leaf nodes make predictions



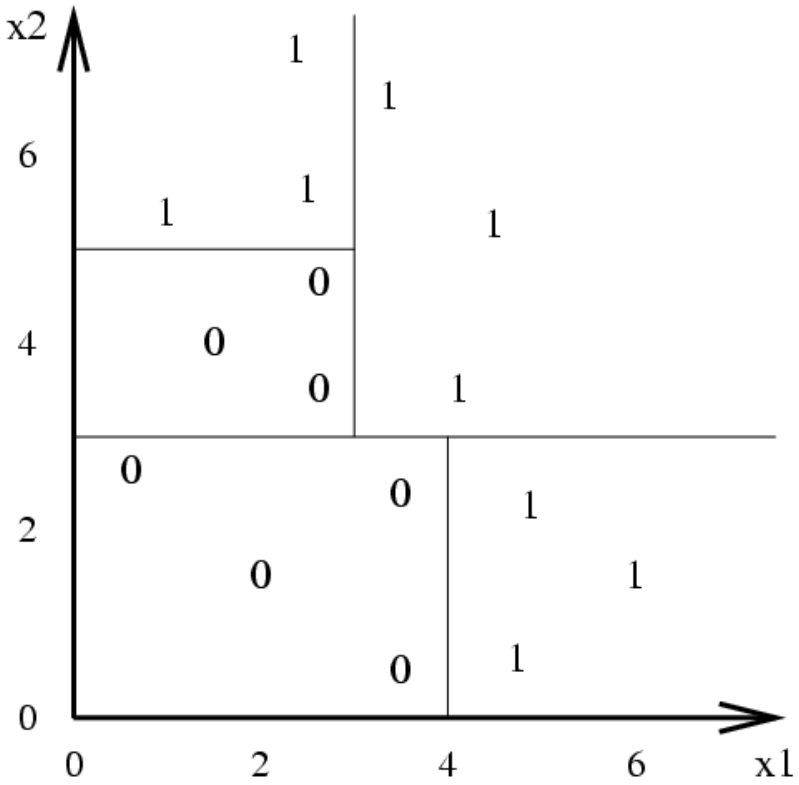
Decision Tree

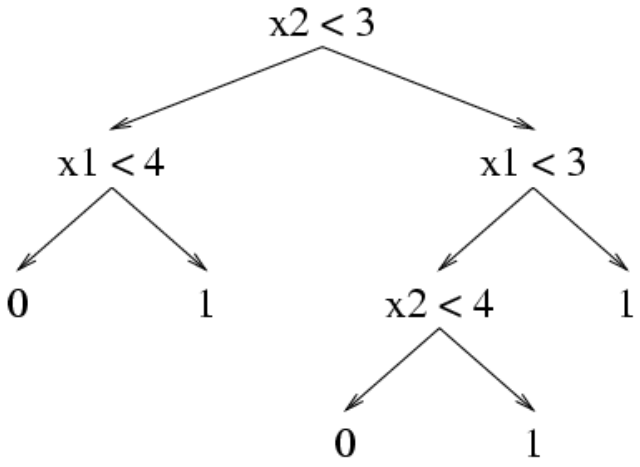
Decision trees divide the features space into axis-parallel rectangles and label each rectangle with on of the k classes





Decision Tree





The tree will in the wort case require many nodes and that grow is exponential

Decision Tree

$$S = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}$$

$$\mathbf{x} = (x_1, \ldots, x_d)$$

$$x_j, y \in \{0,1\}$$

GROWTREE(S)

if ($y = 0$ for all $\langle \mathbf{x}, y \rangle \in S$) **return** new leaf(0)

else if ($y = 1$ for all $\langle \mathbf{x}, y \rangle \in S$) **return** new leaf(1)

else

choose best attribute x_j

How to do this?

$S_0 =$ all $\langle \mathbf{x}, y \rangle \in S$ with $x_j = 0$;

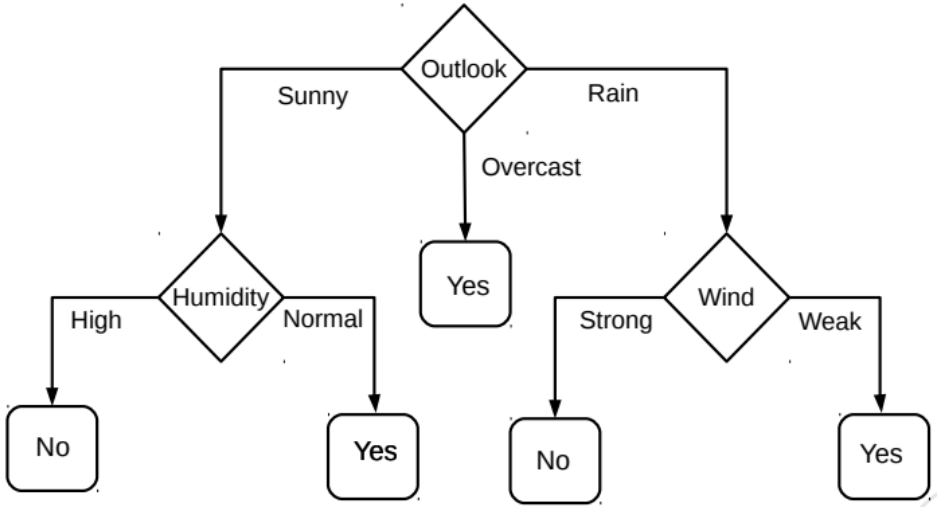
$S_1 =$ all $\langle \mathbf{x}, y \rangle \in S$ with $x_j = 1$;

return new node(x_j, GROWTREE(S_0), GROWTREE(S_1))

Decision Tree

Let's considerer a practical example: Deciding whether to play or not to play Tennis

- 4 categorical features: Outlook, Temp., Humidity, Wind
- A binary classification problem (play vs no-play)



day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no

Decision Tree
Information Gain

First decision, which feature to the root node.

For that, it is necessary some metrics:

- Information Gain - maximize the reduction in entropy impurity or gini impurity

By intuition, in this context, entropy is a measure of 'the degree of surprise'.

- Same dominant class => small entropy (less uncertainty
- Equiprobable classes => high entropy (more uncertainty)

Assume a set of labelled inputs $\boldsymbol{S}$ from C classes, p_c as fraction of class c inputs

Entropy Impurity of the set $\boldsymbol{S}$ is defined as:

$$H(\boldsymbol{S}) = -\sum_{c \in C} p_c \log p_c$$

Suppose a rule splits $\boldsymbol{S}$ into two smaller disjoint sets $\boldsymbol{S}_1$ and $\boldsymbol{S}_2$

Reduction in entropy after the split is called information gain:

$$IG = H(S) - \frac{|S_1|}{|S|} H(S_1) - \frac{|S_2|}{|S|} H(S_2)$$

Decision Tree
Information Gain

day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no

Let assume that **wind** will be the root node:

Root node: S=[9+,5-]

- From all tanning data (S), 9 play and 5 no-play

Entropy of all S

- $H(S) = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right) = 0.94$

Entropy of S_{week}= [6+,2-] => $H(S_{week})$ = 0.811

Entropy of S_{strong}= [3+,3-] => $H(S_{strong})$= 1

Information Gain IG:

- $IG(S, wind) = H(S) - \frac{S_{week}}{S} H(S_{week}) - \frac{S_{strong}}{S} H(S_{strong}) = 0.94 - \frac{8}{14} \cdot 0.811 - \frac{6}{14} \cdot 1 = 0.048$

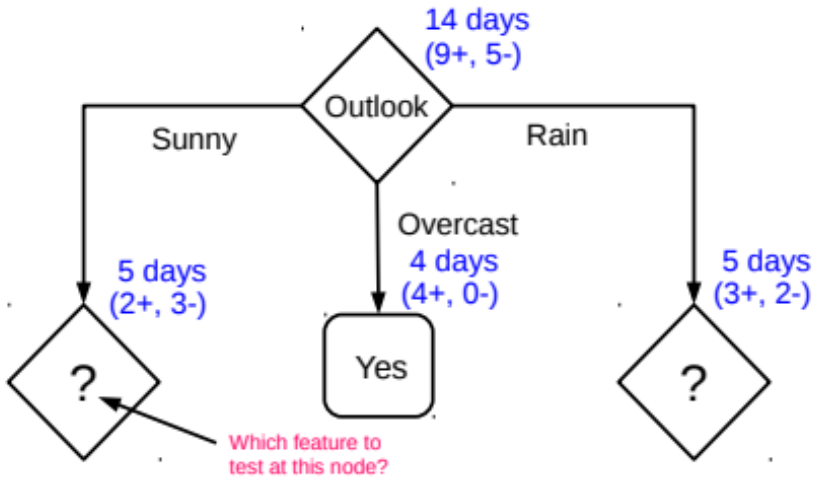
Decision Tree
Information Gain

Calculating the IG for all possible features:

- IG(S, wind) = 0.048
- IG(S, outlook) = 0.246
- IG(S, humidity) = 0.151
- IG(S, temperature) = 0.029

Outlook is the best candidate to be root node

Which feature to select to the next note?



day	outlook	temperature	humidity	wind	play
1	sunny	hot	high	weak	no
2	sunny	hot	high	strong	no
3	overcast	hot	high	weak	yes
4	rain	mild	high	weak	yes
5	rain	cool	normal	weak	yes
6	rain	cool	normal	strong	no
7	overcast	cool	normal	strong	yes
8	sunny	mild	high	weak	no
9	sunny	cool	normal	weak	yes
10	rain	mild	normal	weak	yes
11	sunny	mild	normal	strong	yes
12	overcast	mild	high	strong	yes
13	overcast	hot	normal	weak	yes
14	rain	mild	high	strong	no

Decision Tree
Information Gain

Branch sunny – population (Ssunny)

Sunny node: S_{sunny}=[2+,3-]

- From sunny training data (S_{sunny}), 2 play and 3 no-play

Entropy of all S_{sunny}

- $H(S) = -\left(\frac{2}{5}\right)\log_2\left(\frac{2}{5}\right) - \left(\frac{3}{5}\right)\log_2\left(\frac{3}{5}\right) = 0.970$

Entropy for wind of S_{week}= [1+,2-] => H(S_{week}) = 0.918

Entropy for wind of S_{strong}= [1+,1-] => H(S_{strong})= 1

Entropy for temperature of S_{hot}= [0+,2-] => H(S_{hot}) = 0.

Entropy for temperature of S_{mild}= [1+,1-] => H(S_{mild})= 1

Entropy for temperature of S_{cold}= [1+,0-] => H(S_{cold})= 0

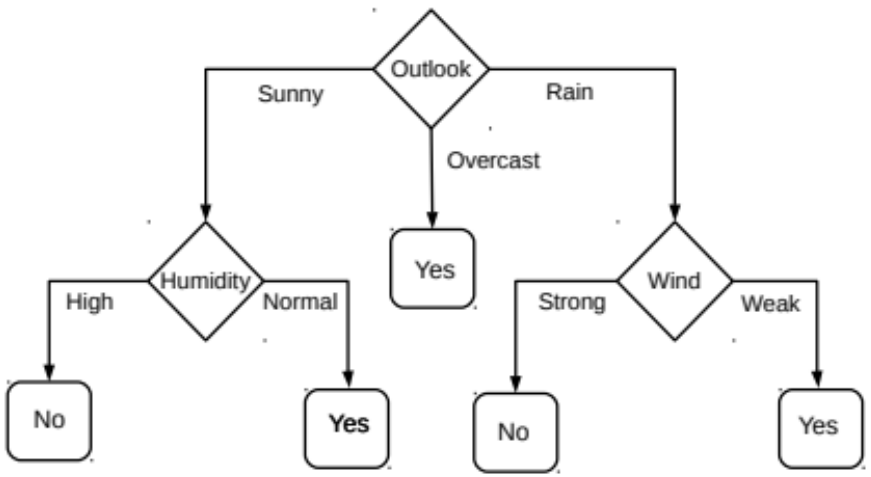
Entropy for humidity of S_{high}= [0+,3-] => H(S_{high}) = 0

Entropy for humiduty of S_{normal}= [1+,1-] => H(S_{normal})= 0

Information Gain IG:

- $IG(S_{sunny}, wind) = 0.97 - \frac{2}{5}\cdot 1 - \frac{3}{5}\cdot 0.918 = 0.019$
- $IG(S_{sunny}, temperature) = 0.97 - \frac{2}{5}\cdot 0 - \frac{1}{5}\cdot 1 - \frac{2}{5}\cdot 0 = 0.570$
- $IG(S_{sunny}, humidity) = 0.97 - \frac{3}{5}\cdot 0 - \frac{2}{5}\cdot 0 = 0.970$

Selection: humidity



Decision Tree
Information Gain

To increase computational performance, often the entropy impurity is replaced by GINI impurity

The calculus of information gain, remains similar

Entropy impurity

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2 \left(p_{i,k}\right)$$

Gini impurity

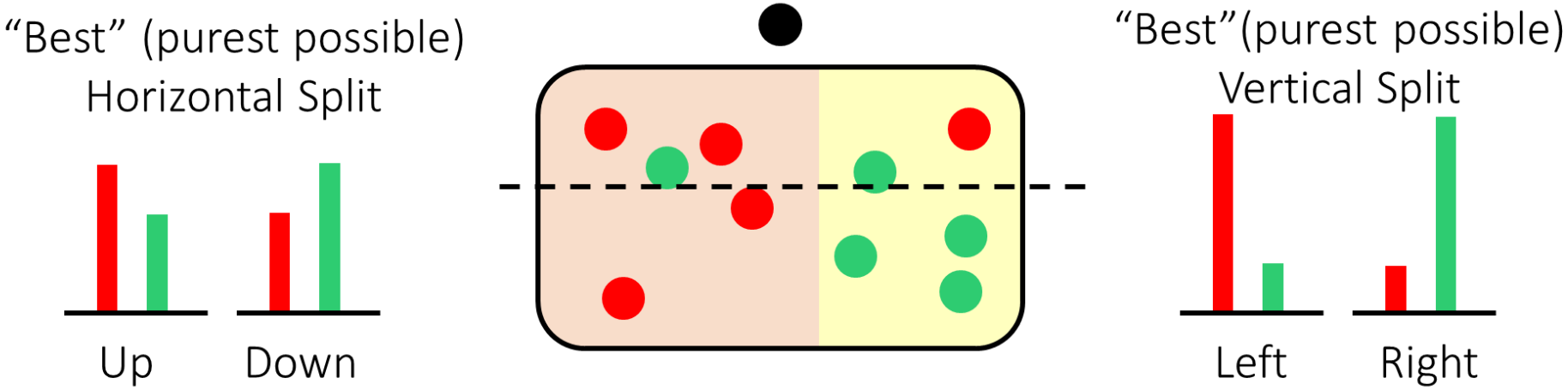
$$G_i = 1 - \sum_{k=1}^{n} {p_{i,k}}^2$$

Decision Tree
Non-Boolean Features

Features with multiple discrete values
- Multi-way splits
- Test for one value versus the rest
- Group values into disjoint sets

Real-valued features
- Use thresholds

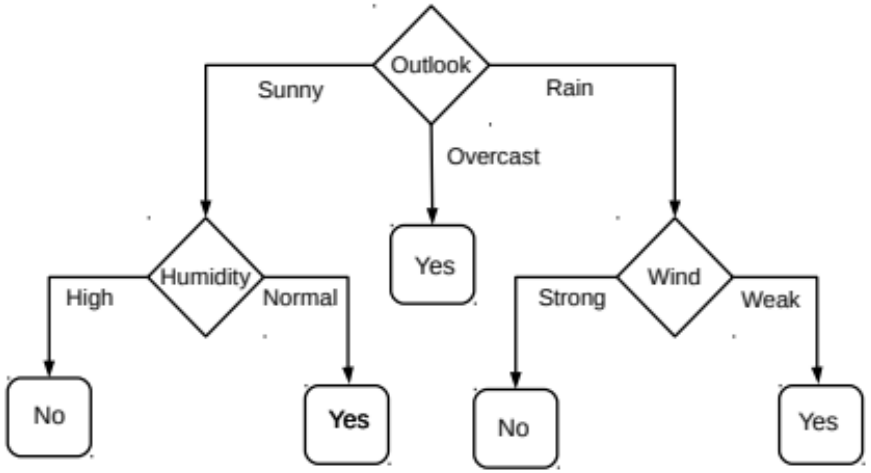
Regression
- Splits based on mean squared error metric

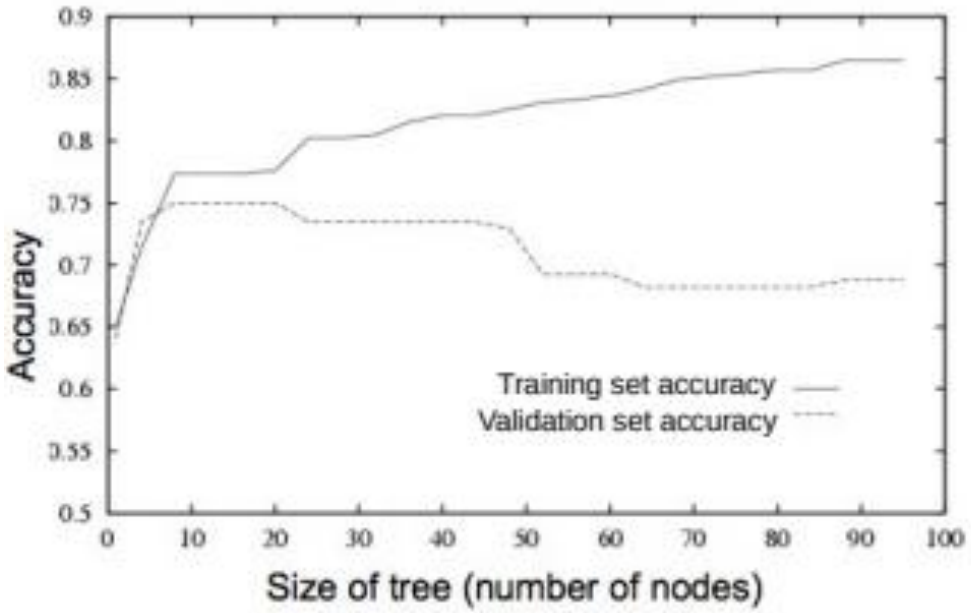


Decision Tree
Stop Criterium

Stop expanding a node further (i.e., make it a leaf node) when

- It consist of all training examples having the same label (the node becomes “pure”)
- We run out of features to test along the path to that node
- The DT starts to overfit (can be checked by monitoring the validation set accuracy)
- The number maximum of branches is reached





KNN – k – Nearest Neighbour

KNN – k – Nearest Neighbour

Idea:

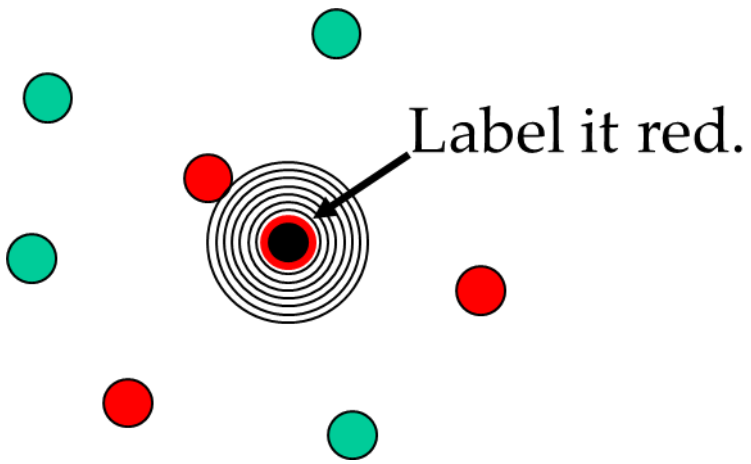
- Similar examples have similar label.
- Classify new examples like similar training examples.

Algorithm:

- Given some new example x for which we need to predict its class y
- Find most similar training examples
- Classify x “like” these most similar examples

Learning

- There is no explicit training phase
- Very fast ‘training’ phase



KNN

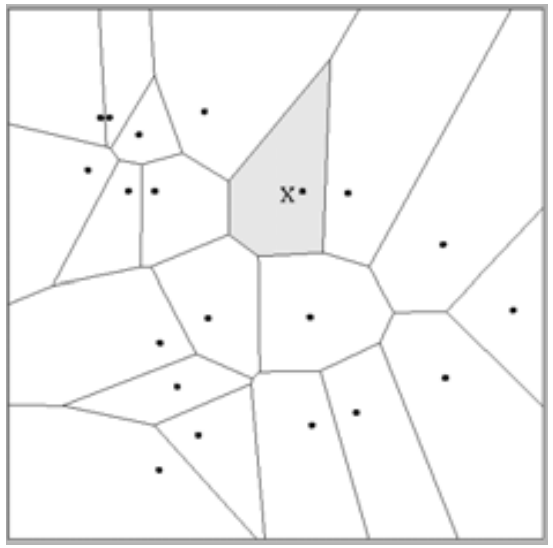
I way often used to visualize the Space, is through Voronoi tessellation diagrams

Sometimes is useful to decide the metrics of distance to use

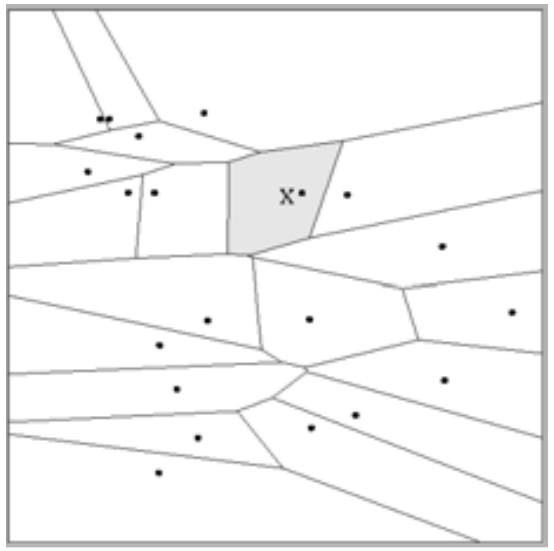
Standard Euclidean distance metric:

- Dist(a,b) = sqrt($\sum (a_i - b_i)^2$)

Other metrics can be used as the Hamming or Manhattan distance



$\text{Dist}(\mathbf{a},\mathbf{b}) = (a_1 - b_1)^2 + (a_2 - b_2)^2$



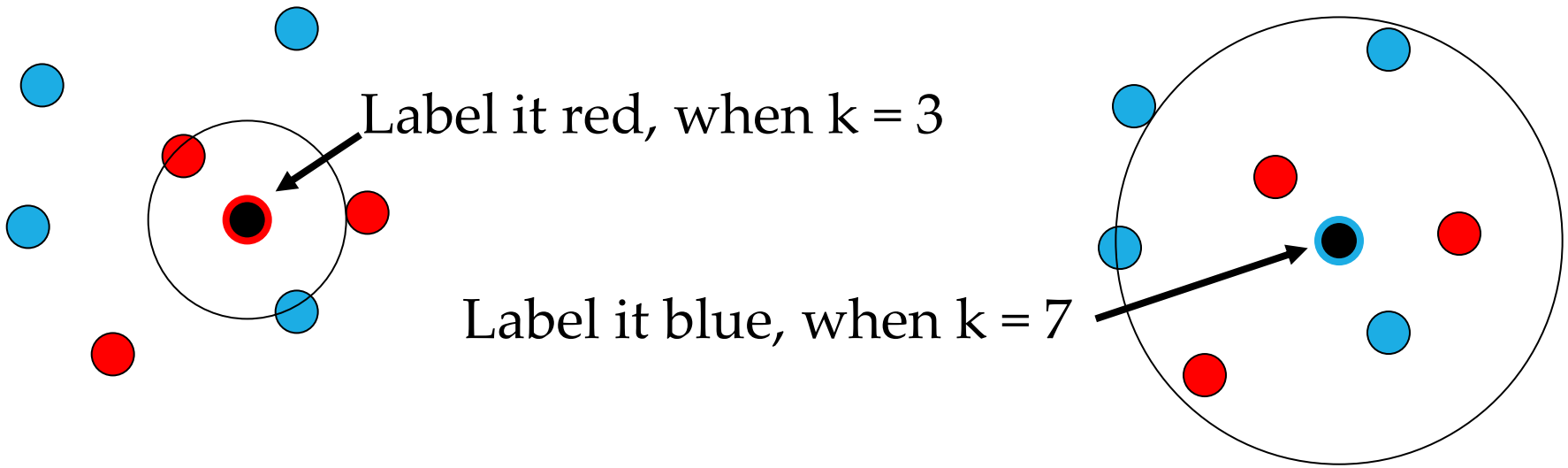
$\text{Dist}(\mathbf{a},\mathbf{b}) = (a_1 - b_1)^2 + (3a_2 - 3b_2)^2$

KNN

Until now, it was always being considered that the first neighbour gives the solution (k=1)

That is not necessary a good idea if the data is noisy or unstable

Solution, increase the value of k



KNN

Increase k:

- Makes KNN less sensitive to noise

Decrease k:

- Allows capturing finer structure of space

Prediction accuracy can quickly degrade when number of features grows.

- Irrelevant attributes easily “swamp” information from relevant attributes
- When many irrelevant attributes, similarity/distance measure becomes less reliable

Solution

- Try to remove irrelevant features in pre-processing step
- Weight features differently
- Increase k (but not too much)

SVM - Support Vector Machine

Basic Idea: Learn how to separate two classes by a hyperplane:

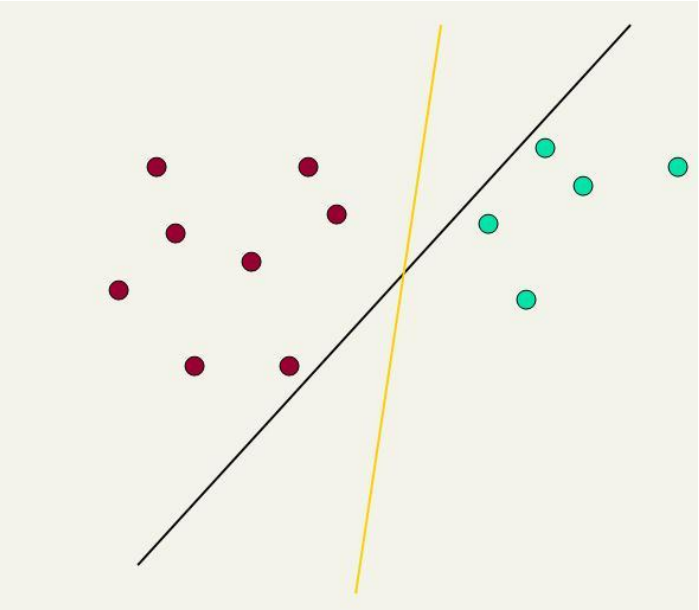
$$\omega^T x + b = 0$$

Where *x* is the features of the data

SVM is one of the techniques to find 'best' hyperplane solution.

- Maximizes the distance between the hyperplane and the "difficult points" close to decision boundary
- One intuition: if there are no points near the decision surface, then there are no very uncertain classification decisions

The base technique find linear hyperplanes, but complementary techniques can be used to other geometries.



Different hyperplanes can be found that separate the two classes

SVM

w: decision hyperplane normal vector

x_i: data point i

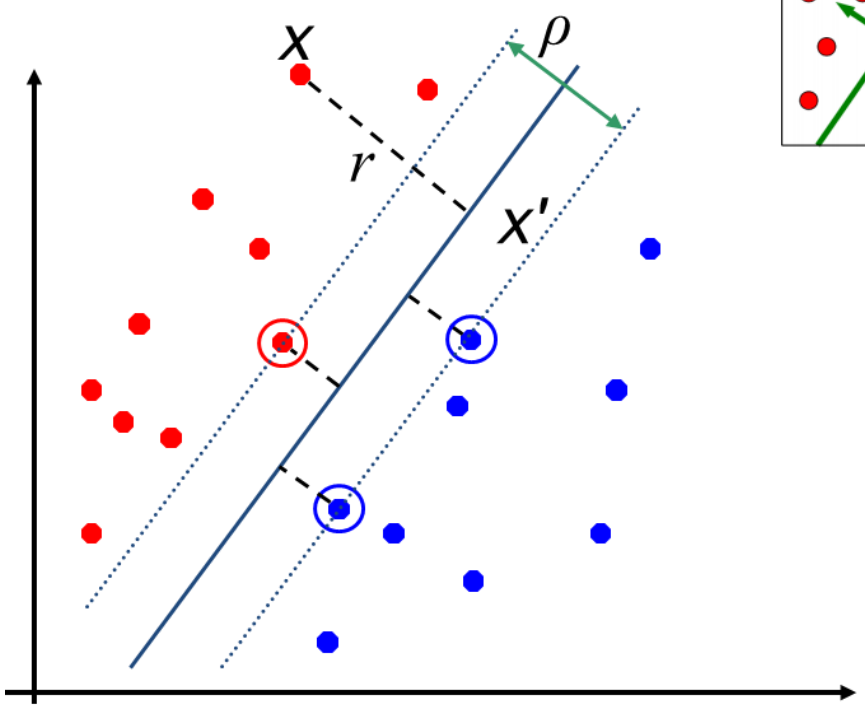
y_i: class of data point i (+1 or -1)

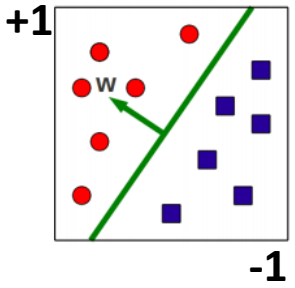
- Note: Not 1/0

Classifier is: $y_i \quad = sign(\omega^T x_i + b\,)$

Examples closest to the hyperplane are the ***support vectors***.

Margin ρ of the separator is the width of separation between support vectors of classes.





SVM

Let's assume that all data is at least distance 1 from the hyperplane, then the following two constraints follow for a training set {(xi ,yi)}

$$\omega^T x_i + b \geq 1 \;\; if \;\; y_i = 1$$

$$\omega^T x_i + b \leq -1 \;\; if \;\; y_i = -1$$

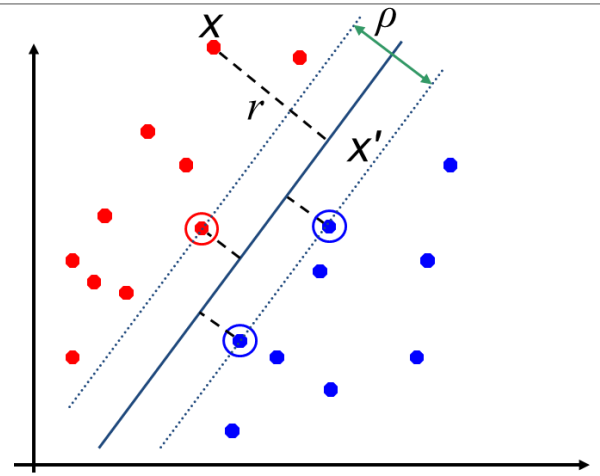
For support vectors, the inequality becomes an equality

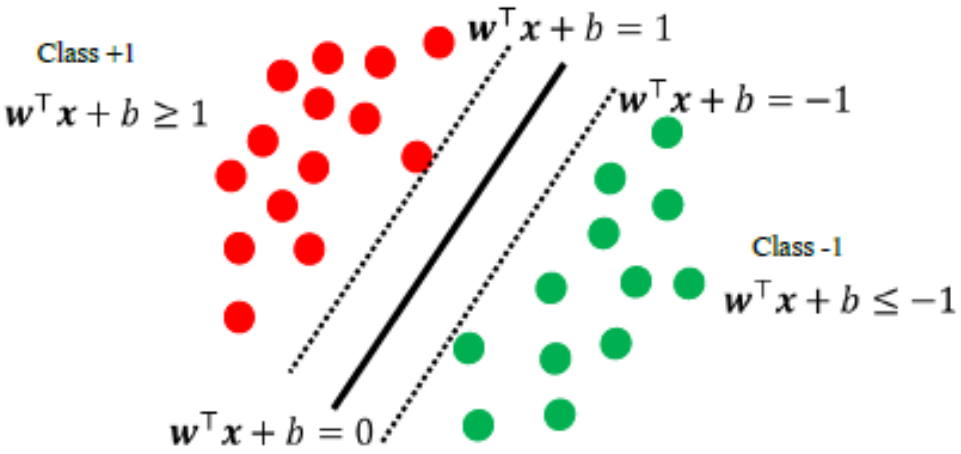
Then, since each example's distance from the hyperplane is

$$r = \frac{\omega^T x \;\; + b}{\|\omega\|}$$

The margin is:

$$\rho = \frac{2}{\|\omega\|}$$





SVM

The SVM goal is:

Find w and b such that:

$$\rho = \frac{2}{\|\omega\|}$$

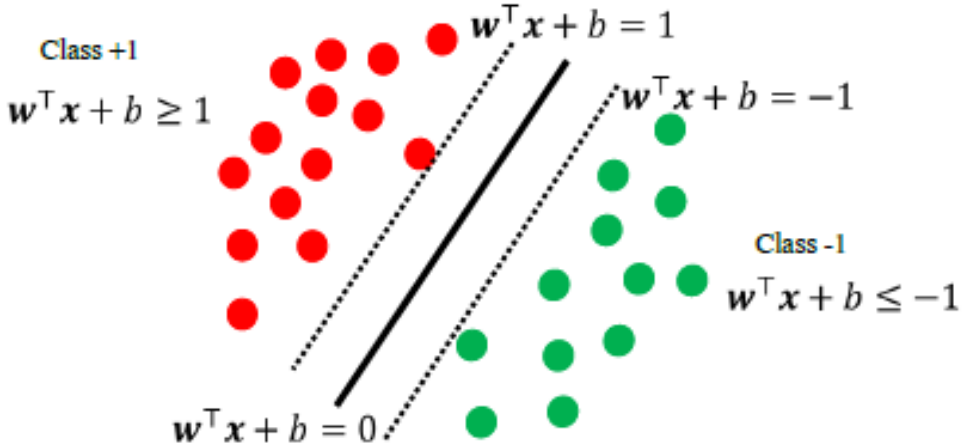
is maximized; and for all $\{(x_i, y_i)\}$

Being verified:

$$\omega^T x_i + b \geq 1 \;\; if \;\; y_i = 1$$

$$\omega^T x_i + b \leq -1 \;\; if \;\; y_i = -1$$

This is a constrained optimization problem and often solved through the Lagrange methods.



Between the two extra hyperplanes no training examples should exist.

- *No man's land*

The SVM's attempts to do this region as wide as possible.

SVM

Hard-Margin SVM

- When all the training values respect the *no man's zone*.

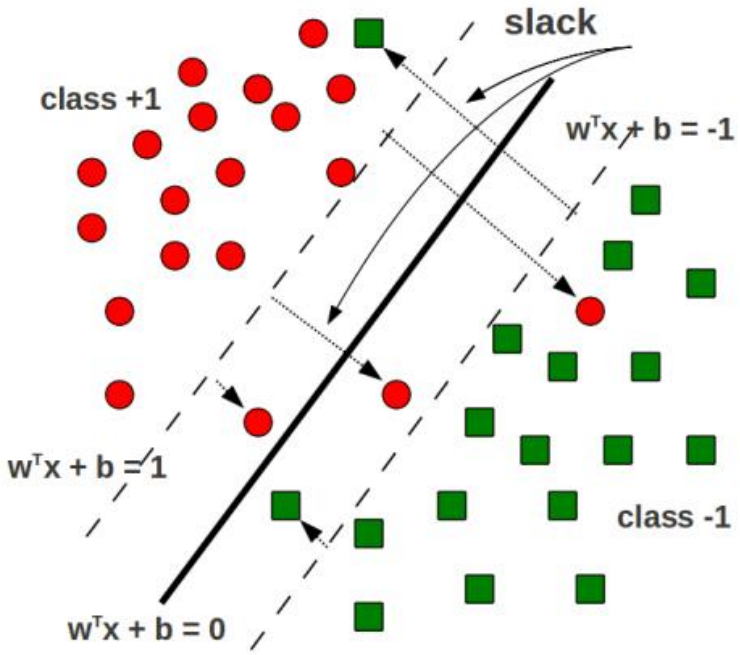
Soft-Margin SVM

- When some training value fall inside the *wrong area*.
- In this situation the position of the training values e correct through a penalization factor.

$$\omega^T x_i + b \geq 1 - \xi_i \;\; if \;\; y_i = 1$$

$$\omega^T x_i + b \leq -1 + \xi_i \; if \;\; y_i = -1$$

- One more condition: $\sum \xi_i$ must be minimized



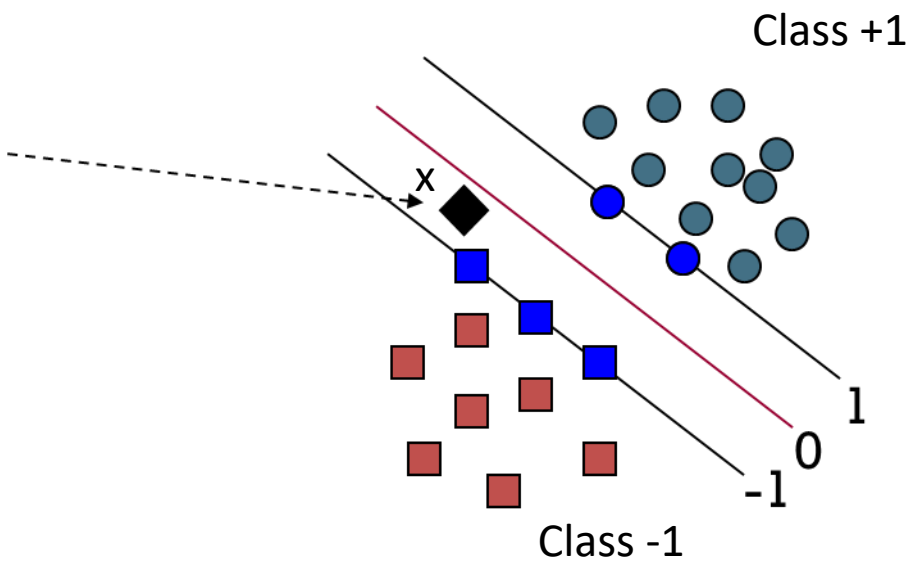
SVM

Classification:

- Given a new point x, we can score its projection with:

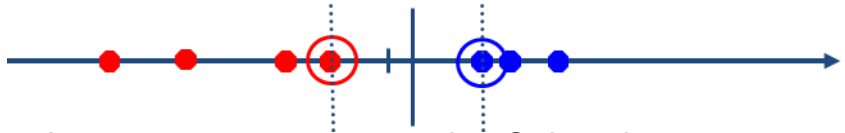
$$y = \omega^T x + b$$

  - If y > 1 belongs to class +1
  - If y < 1 belongs to class -1
  - if 1 < y < 1 in no man's zone. It is set a confidence threshold t
    - If y > t belongs to class +1
    - If y < t belongs to class -1
    - if 1 < t < 1 unknown.



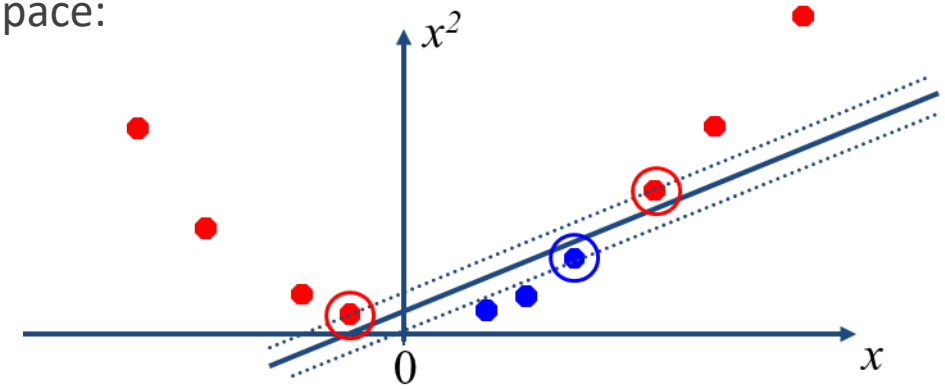
SVM – the Kernel Trick

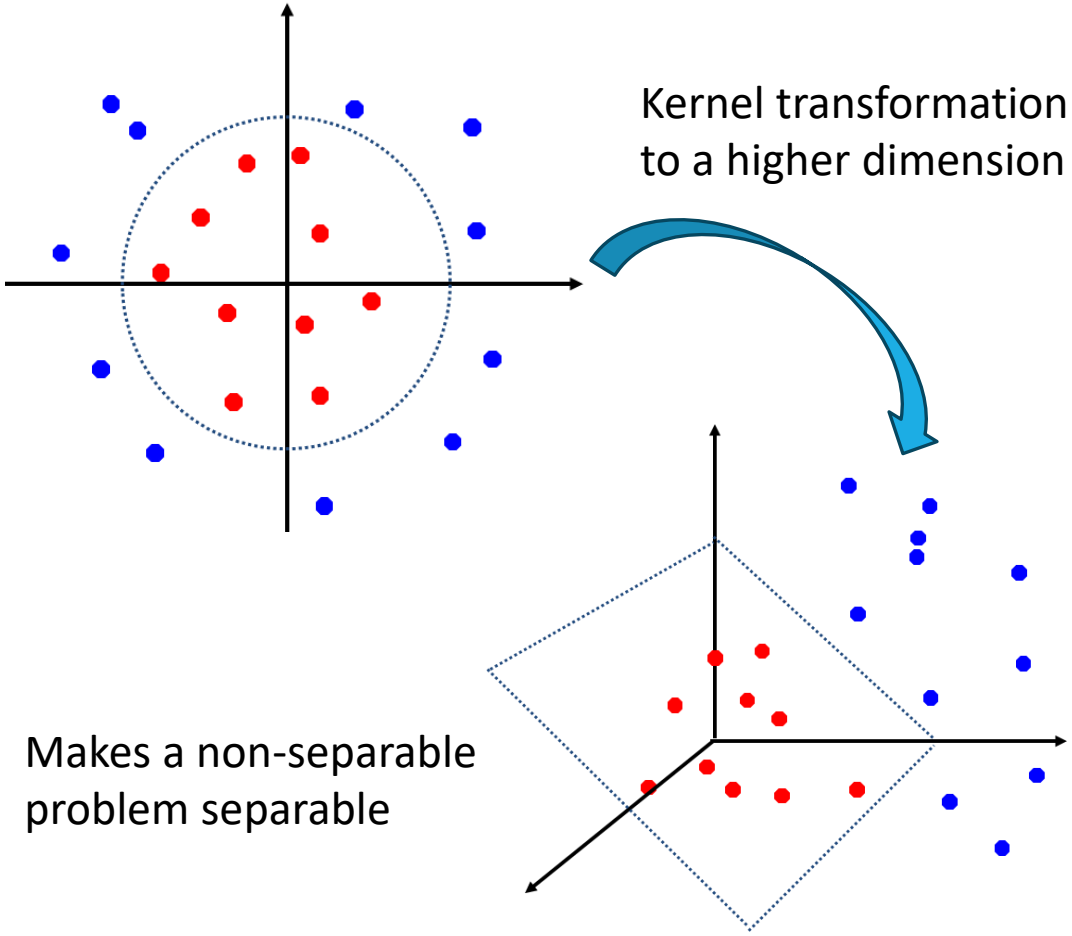
Datasets that are linearly separable (with some noise) work out great:



But what are we going to do if the dataset is just too hard?

How about … mapping data to a higher-dimensional space:





Decision tree vs KNN vs CNN

Algorithm	Do's	Dont's
Decision Trees	• You want interpretability (clear rules) • Data has mixed types (numeric + categorical) • You need a fast model to train	• You have very high-dimensional data (trees overfit) • You need high stability (small changes → different tree)
k-Nearest Neighbours (kNN)	• You want a simple, intuitive model • You have small datasets • The decision boundary is highly irregular	• You have large datasets (slow at prediction) • Features are on different scales (distance breaks) • You have high-dimensional data (curse of dimensionality)
Support Vector Machines (SVMs)	• You need strong performance on medium-sized datasets • Classes are separable (even in higher dimensions) • You want robustness to outliers (with soft margin)	• You have very large datasets (training is slow) • You need probabilities (SVMs don't give them natively) • Data has many irrelevant features (SVMs are sensitive)

Variants to Decision Trees and KNN

Classifiers of the family of decision tree's

- Random Forest
- Gradient Boosted Trees
- Extra Trees (Extremely Randomized Trees)
- Oblique Trees

Classifiers of the family of decision KNN's

- Radius Neighbours Classifier
- Prototype-based Methods
- Distance-metric learning models